ADC and Analog Interfacing

Introduction to ARM Cortex-M Microprocessors — Chapter 11

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

Table of Contents

1 Why This Matters

2 Sampling and Resolution

3 Using the ADC

Why This Chapter Matters

Temperature, light, sound and force are continuous. A processor works only with numbers. The ADC crosses that gap. Using it well is an engineering problem: how fast to sample, how much resolution you need, and how much of the answer you can trust.

After this chapter, you can:

1. Explain sampling, quantization and resolution; apply Nyquist.
2. Calculate the step size and convert codes to volts.
3. Configure a pin for analog input and set up the ADC.
4. Read by polling and by interrupt.
5. Choose the sampling time for a source impedance.
6. Scale a raw reading into engineering units.

© Muním Zabidi

Table of Contents

1 Why This Matters

2 Sampling and Resolution

3 Using the ADC

Two Approximations at Once

Sampling — discrete in time

You measure at instants, not continuously. Between samples, the signal is unknown.

Nyquist: sample at more than twice the highest frequency present, or the reconstruction is wrong.

Quantization — discrete in value

Each sample becomes one of a finite set of numbers. 12 bits gives $2^{12} = 4096$ levels, 0 to 4095.

The error is up to half a step — inherent, not a defect.

Aliasing is not recoverable

Sample a 600 Hz tone at 1 kHz and it reappears as 400 Hz — and no amount of later processing can tell it from a real 400 Hz signal. The fix is an anti-alias filter in hardware, before the ADC.

From Count to Voltage

$$V_{\text{in}} = \frac{\text{count}}{4095} \times V_{\text{ref}}$$

Count	Voltage ($V_{\text{ref}} = 3.3\,\text{V}$)	Meaning
`0`	0.00 V	Input at ground
`1`	0.81 mV	One step — the smallest detectable change
`2048`	1.65 V	Half scale
`4095`	3.30 V	Full scale

Resolution is a voltage, not a bit count

"12-bit" sounds abstract. What it *means* is that the smallest change you can see is $3.3/4095 \approx 0.8\,\text{mV}$. Anything finer is invisible to your program.

© Muním Zabidi

Table of Contents

1 Why This Matters

2 Sampling and Resolution

3 Using the ADC

Configuration

```
// 1. Clocks
RCC->AHB1ENR |= (1U << 0);      // GPIOA
RCC->APB2ENR |= (1U << 8);      // ADC1

// 2. PA0 as analog -- MODER bits 11, the fourth mode from Chapter 8
GPIOA->MODER |= (3U << 0);

// 3. Sampling time: 480 cycles on channel 0
ADC1->SMPR2 |= (7U << 0);

// 4. Sequence: one conversion, channel 0
ADC1->SQR1 = 0;                 // L = 0 means 1 conversion
ADC1->SQR3 = 0;                 // first in sequence is channel 0

// 5. Turn it on
ADC1->CR2 |= (1U << 0);         // ADON
```

On this course `PA0` is `ADC1` channel `0` — and note `MODER` = `11`, the analog mode that has been waiting since Chapter 8.

Reading a Value

```c
uint16_t ADC_Read(void)
{
    ADC1->CR2 |= (1U << 30);   // SWSTART
    while (!(ADC1->SR & (1U << 1)))
        ;                      // wait for EOC
    return ADC1->DR;           // reading DR
}                              // clears EOC
```

1. Start — set `SWSTART`
2. Wait — poll `EOC`
3. Read — `DR` holds the result, and reading it clears the flag

This blocks, and Chapter 9 said why that is a shame

The processor does nothing during the conversion. A real design uses the end-of-conversion interrupt, or lets DMA move the result with no CPU involvement at all.

- The ADC charges an internal capacitor from your signal before converting.
- Sampling time is how long it is given to charge — 3 to 480 cycles.

```
ADC1->SMPR2 |= (7U « 0); // 480 cycles
```

Too short, and you read nonsense

A high-impedance source — a potentiometer, a sensor behind a large resistor — charges that capacitor slowly. Convert too early and you measure a *partly charged* capacitor, not your signal.

The symptom is characteristic: readings consistently low, or changing when you touch the wiring. The fix is a longer sampling time — or a buffer amplifier.

© Munim Zabidi

From Codes to Engineering Units

A thermocouple amplifier

Outputs 0–3.3 V for 0–400 °C.

```
float volts = (raw * 3.3f) / 4095.0f;
float degC  = volts * (400.0f / 3.3f);
```

Or in one step:

```
float degC = raw * (400.0f / 4095.0f);
```

Can you actually resolve 1 °C?

400 °C over 4096 steps is 0.098 °C per step — ten times better than needed.

So the ADC is not your limit. Noise is — and that is an analog design problem, not a programming one.

Note the `f` suffixes: `3.3f` is a `float`; plain `3.3` is a `double`, which the Cortex-M4's FPU cannot do in hardware.

© Muním Zabidi

Resolution Is Not Accuracy

Resolution

The smallest change the ADC can tell apart.

12-bit: $3.3/4096 \approx 0.8$ mV.

A property of the bit count. It never gets better than this.

Accuracy

How close the reading is to the true voltage.

Set by reference tolerance, offset, gain error, noise, source impedance, sampling time.

Can be far worse than resolution. 12-bit numbers do not guarantee 12-bit answers.

Averaging

N independent samples reduce random noise by $\sqrt{N}$. Trades conversion rate for a quieter number.

An Analog Measurement at Five Levels

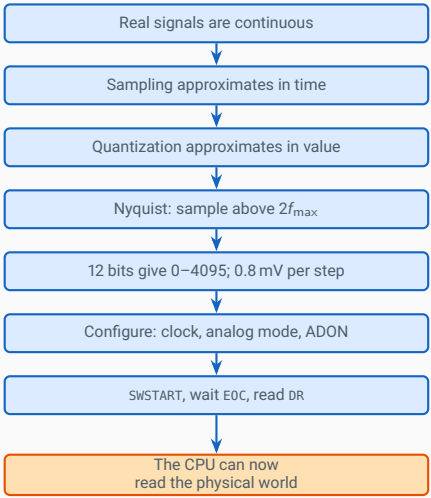
Level	Representation
Physical	1.599 V on the PA0 pad from the potentiometer wiper
Hardware	Sample-and-hold capacitor; 12-cycle SAR comparison
Register	`ADC1->DR` contains `0x7C0`
C	`voltage = raw * 3.3f / 4095.0f;`
Engineering	1.599 V, or 159.9 °C from an LM35

What the ADC gives you

A continuous quantity sensed through a chain that starts at a physical voltage and ends in engineering units. With a hardware timer driving conversions, the interval is set by the peripheral, not by the main loop.

© Muním Zabidi

Summary



Terminology covered

- Sampling, quantization
- Nyquist, aliasing
- Resolution, step size
- Sampling time, impedance
- `EOC`, `DR`

Next: Chapter 12

UART Communication

We can read the world. Next, we report it.

© Muním Zabidi